ELARA: Energy-Efficient and Latency-Aware Microservice Orchestration in Space Computing Power Networks

Anonymous Authors

***Abstract*—The rapid advancement of the Space Computing Power Network (SCPN), comprising numerous low Earth orbit (LEO) satellites, has enabled the execution of complex in-orbit observation and computation tasks. To facilitate this, microservice architecture decomposes large, monolithic remote sensing and image processing applications into independent, schedulable, and reusable services deployed in a distributed manner across LEO satellites. However, integrating these capabilities presents significant challenges arising from the limited computing resources, constrained power provisions of individual satellites, and the dynamic nature of space network topology and channel conditions. This paper addresses these challenges by proposing an energy-aware service routing algorithm for microservices within the SCPN. The algorithm aims to minimize the energy consumed by the procedures of processing and transmitting service data in both computation and communication. By modeling the service routing problem as a Directed Steiner Tree (DST) and implementing heuristic-based solutions, our algorithm effectively responds to the diverse service patterns requested by terrestrial clients. Comprehensive experimental results demonstrate that our approach significantly outperforms existing methods in terms of energy efficiency, robustness, stability, and adaptability.**

***Index Terms*—Energy, Microservice, Routing, Space Network**

I. Introduction

Recent advances in low Earth orbit (LEO) mega-constellations are transforming satellites from bent-pipe communication relays into distributed computing nodes in space. These large-scale constellations, comprising hundreds to thousands of satellites connected by inter-satellite links (ISLs) and equipped with onboard processors and storage resources, enable global connectivity and near-data processing for time-sensitive applications [1], [2]. This evolution is particularly critical for Earth observation, weather forecasting, environmental monitoring, and remote sensing analytics, where massive volumes of images and multi-modal data are continuously generated in orbit [3], [4]. Directly downloading such raw data to terrestrial data centers for analysis will consume scarce satellite-to-ground bandwidth and exacerbate link congestion, especially under limited contact windows and unfavorable channel conditions [5], [6]. As a result, space computing power networks (SCPNs), which pool sensing, communication, and computing resources across LEO satellites, have emerged as a promising infrastructure for onboard intelligent services [7].

Promising but challenging, running these complex monolithic applications on a single LEO satellite remains infeasible due to stringent resource constraints, e.g., limited computing, storage, energy provisions, and thermal dissipation [5], [6]. Microservice architecture provides an effective solution by decomposing these remote sensing and AI applications into fine-grained, loosely coupled, and reusable microservices, each encapsulating a distinct functional module [8]. By packaging them into lightweight containers and deploying them across interconnected satellites, this architecture alleviates individual satellite resource limitations and enables onboard cooperative execution of complex applications. Furthermore, upon detecting hotspot microservices, the system can dynamically scale their replicas and adjust deployment strategies to ensure robust and efficient service provisioning.

Typically, a single request initiated by a terrestrial user involves a subset of microservices. The dynamic and time-varying constellation topology can affect or even interrupt the data transmission process because of unstable inter-plane ISLs. Moreover, the harware heterogeneity and limited onboard power supply further complicate the scheduling decisions at each microservice execution stage.

To enable energy-efficient and low-latency service provision in SCPNs, there are several major challenges must be addressed. First, the satellite network topology is predictable but highly time-varying. Intra-plane ISLs are relatively stable, whereas inter-plane ISLs are sensitive to satellite motion and change frequently across slots. Consequently, routing intermediate data along a single static path often suffers from link disruptions, congestion, or degraded transmission rates. Second, serving satellite selection and ISL routing are tightly coupled. Selecting a nearby replica may reduce transmission delay but lead to long computation queuing time on overloaded satellites, while choosing an idle satellite can incur high multi-hop communication costs. This coupling is further complicated by heterogeneous computing capacities, and energy constraints [9], [10]. Third, microservice replica deployment cannot remain static. As request distributions, satellite workloads, and network topology evolve, previously optimal placements quickly become inefficient. However, frequent redeployment is costly due to expensive service image transfers over ISLs and the enormous decision space [11].

Existing studies have extensively investigated satellite edge computing, in-orbit computation, service placement, and NFV-enabled service chaining in the context of SCPN [5], [6], [11]–[13]. Nevertheless, most of them fail to fully capture the continuous execution of microservice requests and ignore the joint optimization of service routing and replica deployment under time-varying communication conditions and heteroge-

neous onboard hardware environments. In particular, many satellite-oriented service and function execution models rely on unchanged constellation snapshots, implicitly assuming that each task can be completed within a single period during which link states remain unchanged. While this abstraction simplifies modeling and algorithm design, it deviates from practical in-orbit service execution, where data routing, computation, and scheduling could frequently span multiple time slots [14]. Although terrestrial microservice routing studies have explored the joint optimization of service deployment and request routing [15], their assumptions of stable networks, abundant power supply, and fixed edge infrastructure are not directly applicable to the SCPNs.

A. Contributions

To address the aforementioned challenges, we propose ELARA, an energy-efficient and latency-aware microservice orchestration framework for SCPNs. ELARA jointly optimizes routing and deployment decisions for distributed microservice-based applications in spatio-temporally dynamic SCPN environments. Specifically, upon a request arrives at an arbitrary continuous time, ELARA partitions the entire execution process into multiple sequential stages. For each stage, after completing the current service, ELARA first selects the next computing node from satellites hosting replicas of the subsequent microservice, while considering the volume of intermediate data, real-time onboard computing capacities, and current ISL states. ELARA then runs a service routing algorithm based on the source and destination satellites, available bandwidths, and ISL congestion levels, generating a set of parallel paths to accelerate data transmission and improve robustness against link disruptions. Finally, ELARA periodically adjusts microservice deployments according to historical service invocation patterns and real-time computing and communication conditions of the SCPN. Built upon this design, ELARA introduces a continuous-time cross-slot service provisioning mechanism along with an adaptive routing-and-redeployment strategy, enabling joint optimization of end-to-end latency and energy consumption under dynamic constellation topologies and stringent onboard resource constraints.

In summary, the main contributions are as follows:

1) We identify and formulate the microservice-based in-orbit service provisioning problem in SCPNs from an application-layer perspective. In particular, we characterize the intertwined challenges of service routing and replica deployment under time-varying constellation topology, heterogeneous onboard computing capacities, and stringent resource constraints. To address these challenges, we propose ELARA, an energy-efficient and latency-aware orchestration framework for microservice execution and deployment across multiple satellites.
2) We design a learning-based service orchestration algorithm that integrates GNN-enhanced Proximal Policy Optimization (PPO) with time-slot-aware adaptive routing. ELARA employs a graph neural network (GNN) to jointly encode satellite computing states, replica deployment, ISL capacity, and service calling information, enabling the PPO agent to learn effective next-hop serving satellite selection policies that adapt to computation heterogeneity and topology dynamics. For data transfer, it further adopts a time-slot-aware min-cost max-flow algorithm to generate parallel and robust routing paths across dynamic LEO snapshots.
3) We propose a multi-armed bandit-based algorithm for adaptive replica placement. By collecting request patterns over a few past time slots along with current onboard computing and ISL conditions, ELARA dynamically adjusts the number of replicas and deployments of hotspot microservices, whose decisions are made at the orbital level rather than by each individual satellite for reducing learning overhead and decision complexity.
4) We conduct extensive simulations under dynamic LEO constellations and diverse microservice workloads to evaluate ELARA's performance. The results demonstrate that ELARA can effectively reduces end-to-end latency, energy consumption, and deadline violations compared with representative baselines, while validating the benefits of joint service orchestration, adaptive routing, and replica deployment.

The remainder of this paper is organized as follows. Sec. II reviews the related works. Sec. III presents the system models. The problem formulation is introduced in Sec. IV. Sec. V details the proposed ELARA framework. Sec. **??** shows and analyses the Simulation results. Sec. VI draws the conclusion.

II. Related Work

Recent advancements in satellite edge computing and SCPNs have supported in-situ data processing over LEO constellations with onboard computation and ISL connectivity [7], [16]. Zhang et al. [17] investigated energy-efficient peer offloading among satellites, while Leyva-Mayorga et al. [5] and Zhai et al. [4] explored real-time Earth observation and joint optimization of observation scheduling, routing, and computation node selection, respectively. These studies primarily focus on task-level offloading, mission-specific processing, or joint computation-communication resource allocation. However, microservice-oriented service provisioning introduces finer-grained execution dependencies across distributed replicas, where intermediate data transmission and onboard computation are tightly coupled under the dynamic SCPN environment.

At the same time, service routing and task scheduling in satellite networks have been extensively studied from various of perspectives. Cao et al. [10] proposed a computing-aware routing framework that jointly optimizes transmission and computation decisions for remote sensing tasks in LEO satellite networks. Xia et al. [12] investigated service chaining in NFV-enabled satellite edge networks. To improve routing efficiency and alleviate link congestion, He et al. [13] developed a throughput-oriented routing scheme for microservice flows. Some studies further model dynamic topologies using

time-expanded graphs or optimize routing over time-varying LEO constellations [14], [18], [19]. However, most of these works either focus solely on communication path optimization or consider computation node selection within static topology snapshots or predefined sequences. They fail to capture the joint impact of dynamic inter-plane ISL disruptions, heterogeneous serving-node computing capacities, and time-varying network topology on microservice requests.

Microservice deployment and request execution have been extensively studied in terrestrial edge systems. Peng et al. [15] proposed efficient algorithms for joint service deployment and request routing in mobile edge computing. In satellite environments, Gao et al. [20] investigated joint server and service selection in satellite-terrestrial integrated edge computing networks, and Yu et al. [11] developed a robust reinforcement learning approach for microservice deployment in SCPNs. Nevertheless, existing methods are either designed for static terrestrial scenarios and thus unsuitable for time-varying SCPNs, or result in prohibitively large joint decision spaces for satellite-service co-optimization. Consequently, effectively coordinating microservice deployment with service routing under dynamic satellite computing conditions and time-varying ISL states remains an open challenge.

III. System Model

We consider an SCPN built upon a Walker-Delta LEO constellation, where satellites are connected through time-varying laser ISLs and equipped with constrained onboard computing and storage resources. Microservice-based applications are deployed as stateless replicas across satellites, allowing each request to be collaboratively executed through successive communication and computation stages. This section models the constellation topology, ISL communication, onboard computation, microservice requests, cross-slot execution cost, and replica migration process.

A. LEO Constellation and Time-slotted Topology

A typical Walker-Delta constellation is parameterized by $\mathcal{W} = (A/P/G, H, I)$, where A is the total number of satellites, P is the number of orbital planes, G is the inter-plane phasing factor, H is the orbital altitude, and I is the inclination. Consequently, each orbital plane accommodates $N = A/P$ satellites. The set of satellite nodes in the constellation is formally defined as $\mathcal{V} = \{v_{p,n} \mid p = 1, \ldots, P,\ n = 1, \ldots, N\}$, where p denotes the orbital plane index, n denotes the intra-plane position index, To capture the high dynamism of the satellite network, we discretize the constellation's orbital period into a set of time slots, as $\mathcal{T} = \{0, 1, \ldots, T-1\}$, with a uniform slot duration Δ. For any continuous-time instant τ, the corresponding time slot index is mapped as, $s(\tau) = \lfloor \tau/\Delta \rfloor \bmod T$. Adopting a snapshot-based modeling approach, we assume that the network state, including satellite positions, ISL availability, and capacity, remains unchanged within a single time slot and only evolves while the slot is switching. Thus, the time-varying network topology during slot $t \in \mathcal{T}$ is formally defined as a dynamic and time-dependent graph $\mathcal{G}(t) = \big(\mathcal{V}, \mathcal{E}(t)\big)$, where $\mathcal{E}(t)$ represents the set of feasible ISLs at slot t.

B. ISL Communication Model

We assume that each satellite is equipped with four laser optical terminals: two positioned along the roll axis for intra-plane ISLs, and two along the pitch axis for inter-plane ISLs.

a) ISL Connectivity Constraints: Each satellite $v_{p,n}$ maintains two intra-plane ISLs with its adjacent neighbors v_{p,n^-} and v_{p,n^+}, where $n^{\pm} = ((n \pm 1) \bmod N) + 1$. Because satellites in the same plane share identical angular velocities, their relative distances remain time-invariant, yielding highly stable communication links.

Conversely, inter-plane ISLs are dynamic. Constrained by the physical limits of onboard laser communication terminals (LCTs), the existence of a inter-plane link $e = (u, v)$ at slot t is jointly affected by line-of-sight (LoS) distance, mechanical tracking thresholds, and polar exclusions, formulated as:

$$e \in \mathcal{E}(t) \iff \begin{cases} d_{u,v}(t) \le d_{\max}, \\ \theta_{u,v}(t) \le \theta_{\max}, \\ \max\big(|\varphi_u(t)|, |\varphi_v(t)|\big) \le \varphi_{\max}. \end{cases} \tag{1}$$

where the LoS distance $d_{u,v}(t)$ and relative tracking angle $\theta_{u,v}(t)$ are bounded by the terrestrial occlusion limit $d_{\max}$ and the hardware threshold $\theta_{\max}$, respectively. Furthermore, the third constraint enforces a polar exclusion zone at latitude boundary $\varphi_{\max}$ (e.g., 70°) through proactively tearing down the inter-plane links, which effectively circumvents the severe pointing dynamics and extreme Doppler shifts inherent to polar regions.

b) ISL Capacity Model: Unlike terrestrial networks with stable capacities, the transmission rate of laser ISLs is distance-dependent due to geometric propagation loss. Based on the Free-Space Optical (FSO) communication model, the received power $P_e^{\text{rx}}(t)$ at satellite v transmitted from satellite u is formalized as:

$$P_e^{\text{rx}}(t) = P^{\text{tx}} G^{\text{tx}} G^{\text{rx}} \eta_{\text{tx}} \eta_{\text{rx}} \left(\frac{\lambda}{4\pi d_{u,v}(t)} \right)^2, \tag{2}$$

where P^{tx} is the laser transmit power, G^{tx} and G^{rx} represent the optical gains of the transmitter and receiver apertures respectively, λ is the laser wavelength, and $\eta_{\text{tx}}, \eta_{\text{rx}}$ denote the optical efficiency factors. The real-time Signal-to-Noise Ratio (SNR) of the link e at time slot t is given as:

$$\gamma_e(t) = \frac{\left(R_{\text{pd}} \cdot P_e^{\text{rx}}(t)\right)^2}{\sigma_{\text{sh}}^2 + \sigma_{\text{th}}^2}, \tag{3}$$

where R_{pd} is the photodiode responsivity, while σ_{sh}^2 and σ_{th}^2 denote the shot and thermal noise variances, respectively. According to the Shannon-Hartley theorem, the achievable transmission capacity $R_e^0(t)$ is constrained by:

$$R_e^0(t) = \mathrm{B}_e \log_2\left(1 + \gamma_e(t)\right), \tag{4}$$

where B_e is the modulation bandwidth.

c) ISL Delay Model: The ideal rate $R_e^0(t)$ in (4) could be degraded in practice by transient channel fading and traffic contention. To capture these factors, we introduce a scaling factor $\eta_e(t) \in (0,1]$ to formulate the effective data rate as $R_e(t) = \eta_e(t) R_e^0(t)$. When transmitting a data block of size B over link $e = (u, v)$ during slot t, the aggregate link-level communication delay comprises queuing, transmission, and propagation delays, formulated as:

$$T_e^{\text{cm}}(B,t) = \nu_e^{\text{qe}}(t) + \nu_e^{\text{tr}}(t) + \nu_e^{\text{pr}}(t). \tag{5}$$

Here, $\nu_e^{\text{qe}}(t)$ denotes the queuing delay, which can be dynamically estimated via lightweight queue-length probing. The transmission delay is given by $\nu_e^{\text{tr}}(t) = B/R_e(t)$, and the propagation delay is $\nu_e^{\text{pr}}(t) = d_e(t)/c$, with c being the speed of light in vacuum.

C. Onboard Computing Model

Let F_v^0 denote the ideal computing capacity of satellite v. In practice, the available computational resources are often constrained by background payload tasks, operating system overhead, and thermal throttling. To capture these hardware and software limitations, we introduce a computational efficiency factor $\xi_v(t) \in (0,1]$ and define the effective dynamic computing capacity as $F_v(t) = \xi_v(t) F_v^0$.

Let $\nu_v^{\text{qe}}(t)$ denote the queuing delay before scheduling microservice m is executed on satellite v, which increases with the pending task queue length at slot t. The total computation delay is modeled as:

$$T_m^{\text{cp}}(v,t) = \nu_v^{\text{qe}}(t) + \nu_v^{\text{cp}}(t), \tag{6}$$

where $\nu_v^{\text{cp}}(t) = C_m / F_v(t)$ represents the actual execution delay, and C_m is the number of CPU cycles required by microservice m. Correspondingly, the computational energy consumption is defined as:

$$E_m^{\text{cp}}(v,t) = P_v^{\text{cp}} \frac{C_m}{F_v(t)}, \tag{7}$$

where P_v^{cp} is the operational computing power of the satellite.

D. LEO Microservice and Service Request Models

To facilitate flexible application management within the SCPN, complex applications are decoupled into a set of fine-grained, loosely coupled microservices, denoted by $\mathcal{M} = \{1, 2, \ldots, M\}$. Each microservice $m \in \mathcal{M}$ is characterized by its computational demand C_m (in CPU cycles), container image size I_m, and storage requirement S_m.

We define a binary indication variable $x_{m,v}(t) \in \{0,1\}$, which equals to 1 if m is deployed on v during slot t. Accordingly, the subset of satellite nodes hosting a valid replica of microservice m at slot t is denoted as $\mathcal{R}_m(t) = \{v \in \mathcal{V} \mid x_{m,v}(t) = 1\}$. To ensure service availability while preventing excessive resource contention, the total number of deployed replicas for any microservice should be bounded within predefined thresholds, $r_m^{\min}$ and $r_m^{\max}$:

$$r_m^{\min} \le \sum_{v \in \mathcal{V}} x_{m,v}(t) \le r_m^{\max}, \quad \forall m \in \mathcal{M}. \tag{8}$$

Furthermore, the aggregate storage footprint of all replicas hosted on a single satellite must not exceed its onboard storage capacity $S_v^{\max}$:

$$\sum_{m \in \mathcal{M}} x_{m,v}(t) S_m \le S_v^{\max}, \quad \forall v \in \mathcal{V}. \tag{9}$$

Let $r = \langle v_s, m_1, m_2, \ldots, m_L, v_d, \tau_{\text{in}} \rangle$ denote an incoming request with a deadline Z_r. Here, v_s and v_d are the source and destination satellites, m_i is the i-th microservice in the sequential execution chain, and τ_{in} is the continuous arrival time. Let $B_{r,i}$ be the data volume that should be transmitted before invoking m_i, while $B_{r,L+1}$ denotes the final output routed to v_d.

a) Cross-slot Service Routing Latency and Energy Model: Since requests may arrive at arbitrary instants while ISL topology and rates change at slot boundaries, ELARA tracks each service chain in continuous time. Before executing m_i, the input data $B_{r,i}$ stored at satellite $u_{r,i}$ should be routed to a satellite hosting a valid replica of m_i. We introduce a binary decision variable $y_{r,i,v} \in \{0,1\}$ for node selection, which equals 1 if m_i is scheduled to satellite v. Let $v_{r,i}$ be the selected satellite with $y_{r,i,v_{r,i}} = 1$. For the i-th service routing operation, the source and destination satellites are denoted by $u_{r,i}$ and $v_{r,i}$, respectively.

Since the endpoints may be non-adjacent and transmission may start near a slot boundary, one routing operation can span $H \ge 1$ phases over time-varying multi-hop routes. For phase h, let $\tau_{r,i}^h$ be its start time, $t_h = s(\tau_{r,i}^h)$ the active topology slot, $\bar{\Delta}_h$ the remaining slot time, and $B_{r,i}^h$ the residual data, with $B_{r,i}^1 = B_{r,i}$. Let $\mathcal{P}_{r,i}^h$ be the feasible and active end-to-end path set from $u_{r,i}$ to $v_{r,i}$ in slot t_h, where each path $p \in \mathcal{P}_{r,i}^h$ is an ordered sequence of one or more ISLs, i.e., $p = (e_1, \ldots, e_{|p|})$, and different path may partially share the same ISLs. If D_h bits are transmitted, ELARA splits them across multiple such paths, and path p receives fraction $\lambda_p^h \in [0,1]$:

$$d_p^h = \lambda_p^h D_h, \qquad \sum_{p \in \mathcal{P}_{r,i}^h} \lambda_p^h = 1, \qquad 0 < D_h \le B_{r,i}^h. \tag{10}$$

The aggregate load on shared ISL $e \in p$ is formulated by:

$$f_e^h = \sum_{p \in \mathcal{P}_{r,i}^h : e \in p} d_p^h, \qquad 0 \le f_e^h \le R_e(t_h) \bar{\Delta}_h. \tag{11}$$

For a multi-hop path p, the subflow completion time follows the link-delay model in (5):

$$\delta_p^h = \sum_{e \in p} T_e^{\text{cm}}(d_p^h, t_h), \qquad \forall p \in \mathcal{P}_{r,i}^h. \tag{12}$$

Since the selected paths transmit in parallel, the effective phase duration is determined by the slowest active path:

$$\Delta_h^{\text{ru}} = \max_{p \in \mathcal{P}_{r,i}^h} \delta_p^h, \quad \text{and,} \quad \Delta_h^{\text{ru}} \le \bar{\Delta}_h. \tag{13}$$

The phase energy is the sum of transmission energy over all traffic-carrying ISLs:

$$E_{r,i}^{h,\text{ru}} = \sum_{e \in \mathcal{E}(t_h)} P_e^{\text{tx}} \frac{f_e^h}{R_e(t_h)}. \tag{14}$$

After phase h, the residual data and decision epoch are updated as:

$$B_{r,i}^{h+1} = B_{r,i}^{h} - D_h, \qquad \tau_{r,i}^{h+1} = \tau_{r,i}^{h} + \Delta_h^{\text{ru}}. \tag{15}$$

If $B_{r,i}^{h+1} > 0$, the next phase is solved under the updated topology, path set, and link rates. For $H_{r,i}$ phases, the routing latency and energy from $u_{r,i}$ to $v_{r,i}$ are:

$$T_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} \Delta_h^{\text{ru}}, \qquad E_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} E_{r,i}^{h,\text{ru}}, \tag{16}$$

The selected replica receives the data at $\tau_{r,i}^{\text{arr}} = \tau_{r,i} + T_{r,i}^{\text{ru}}$. The selected satellite then executes m_i, whose computation delay and energy are given by (6) and (7), respectively.

The cumulative execution latency and energy of request r are given by:

$$T_r^{\text{e2e}} = \sum_{i=1}^{L+1} T_{r,i}^{\text{ru}} + \sum_{i=1}^{L} T_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{17}$$

$$E_r^{\text{tot}} = \sum_{i=1}^{L+1} E_{r,i}^{\text{ru}} + \sum_{i=1}^{L} E_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{18}$$

where $i = L+1$ denotes the final output delivery to v_d.

b) Service Replica Migration Model: Let $\mathbf{X}(t) = [x_{m,v}(t)]$ represent the microservice replica deployment matrix at slot t. ElARA periodically updates the deployment matrix every few time slots by selecting an action $a_{m_i} \in \{\text{add}, \text{remove}, \text{move}, \text{no-op}\}$ for each m_i in a microservice subset $\mathcal{M}'(t) \subset \mathcal{M}$ at time slot t. Then we have:

$$\mathbf{X}(t+1) = \Phi(\mathbf{X}(t), \mathcal{A}_t), \tag{19}$$

where $\Phi(\cdot)$ denotes the deployment transition induced by the selected action set $\mathcal{A}_t = \{a_{m_i} | \forall m_i \in \mathcal{M}'(t)\}$. All redeployment actions should preserve the replica-count and storage constraints in (8) and (9).

IV. Problem Formulation of ELARA

We formulate ELARA as an online energy- and latency-aware service routing problem over a dynamic SCPN. The objective is to simultaneously minimize the end-to-end execution latency and total energy consumption of service chains, including both communication and computation costs, while accounting for the overhead of replica redeployment.

A. Problem Formulation

For each incoming request, ELARA first selects the serving satellite v for each microservice m_i, which satisfies:

$$\sum_{v \in \mathcal{V}} y_{r,i,v} = 1, \quad y_{r,i,v} \le x_{m_i,v}\big(s(\tau_{r,i})\big), \quad \forall r, i, v. \tag{20}$$

Let $v_{r,i}$ denote the selected satellite satisfying $y_{r,i,v_{r,i}} = 1$. ELARA then makes the cross-slot routing policy for service-routing segment i:

$$\Pi_{r,i}^{\text{ru}} = \{D_h, \lambda_p^h, f_e^h, \Delta_h^{\text{ru}}\}_{h=1}^{H_{r,i}}, \tag{21}$$

which follows the multi-hop routing model in Sec. III-D. Additionally, ELARA periodically updates the deployment matrix by applying $\mathcal{A}(t), \forall t \in \mathcal{T}'$, where $\mathcal{T}' \subset \mathcal{T}$.

Following (17) and (18), given an online request r, the request-level optimization objective is defined as:

$$\begin{aligned} \mathscr{P}: \quad \underset{y_{r,v}, \Pi_r^{\text{ru}}, \mathcal{A}}{\text{minimize}} \quad & J_r = \alpha T_r^{\text{e2e}} + \beta E_r^{\text{tot}} + \rho \mathbb{I}_r^{\text{fail}} \\ \text{subject to} \quad & (20), (21), && \text{(C1)} \\ & (10), (11), (13), (15), && \text{(C2)} \\ & (8), (9), (19), && \text{(C3)} \end{aligned}$$

where α, β, and ρ are non-negative weights, and $\mathbb{I}_r^{\text{fail}}$ indicates whether request r violates its deadline. Constraint C1 enforces serving satellite selection by assigning each microservice stage to exactly one satellite hosting a valid replica, and defining the corresponding routing policy for each service routing segment. Constraint C2 ensures feasible cross-slot multi-hop routing under time-varying topology by enforcing path splitting, shared ISL capacity, phase duration, and residual data update constraints. Constraint C3 restricts adaptive replica redeployment to states that satisfy both replica-count and onboard storage constraints after each migration action.

V. The Proposed ELARA Framework

This section presents ELARA, an online orchestration framework that coordinates service routing and replica deployment in dynamic SCPNs. For each arriving request, ELARA selects a serving satellite for every microservice invocation and delivers intermediate data over time-varying multi-hop ISLs. It further adapts replica placement periodically according to recent execution statistics, routing failures, and resource conditions. All decisions share the latency and energy models in Sec. III, enabling coordinated optimization of node selection, data routing, and replica migration.

A. GNN-PPO based Node Selection Agent

We formulate the sequential selection of serving satellites as a Markov Decision Process (MDP), wherein a complete episode corresponds to the end-to-end execution of a single service request. To effectively navigate the time-varying constellation topology and heterogeneous computational resources, ELARA employs a Graph Neural Network (GNN) enhanced Proximal Policy Optimization (PPO) agent to learn the optimal node selection policy.

a) State Space and GNN Representation: For a given request $r = \langle v_s, m_1, \ldots, m_L, v_d, \tau_{\text{in}} \rangle$, the i-th decision epoch is triggered when the required input data for microservice m_i is ready on satellite $u_{r,i}$ at continuous time $\tau_{r,i}$. The system state is defined as a tuple:

$$s_{r,i} = (\mathcal{G}^a(t), u_{r,i}, v_d, \mathcal{M}_{r,i:L}, B_{r,i:L}) \tag{23}$$

where $t = s(\tau_{r,i})$ is the current decision slot, $\mathcal{M}_{r,i:L}$ and $B_{r,i:L}$ represent the sequence of pending service invocations and their corresponding data volumes, respectively. Here, $\mathcal{G}^a(t)$ is an attributed SCPN graph defined as

$$\mathcal{G}^a(t) = \big(\mathcal{V}, \mathcal{E}(t), \mathcal{Q}^v(t), \mathcal{Q}^e(t)\big), \tag{24}$$

where $\mathcal{Q}^v(t)$ denotes node states, including CPU capacity, computing queue length, service deployment, compute utilization, load state, and capacity discount factor. Similarly, $\mathcal{Q}^e(t)$ captures link states, including ISL availability, effective transmission rate, link queue length, and propagation delay within the current time slot.

To enable the PPO agent to process the complex graph states, ELARA encodes each satellite v with a 12-dimensional feature vector $x_v(t)$ for the current attributed graph $\mathcal{G}^a(t)$:

$$\begin{aligned}\mathbf{x}_v(t) = [&\bar{f}_v, \bar{q}_v(t), \bar{s}_v(t), \bar{p}_v, \bar{n}_v, \mathbb{I}_{v=u_{r,i}}, \mathbb{I}_{v=v_d}, \\ &\mathbb{I}_{m_i \in v}, \bar{d}_v(t), \mu_v(t), \xi_v(t), \ell_v(t)].\end{aligned} \tag{25}$$

where $\bar{f}_v$ is the normalized CPU frequency, $\bar{q}_v(t)$ is the compute queueing delay, $\bar{s}_v(t)$ is the normalized number of hosted services, $\bar{p}_v$ is the orbital-plane index, $\bar{n}_v$ is the intra-plane relative position, $\mathbb{I}_{v=u_{r,i}}$ and $\mathbb{I}_{v=v_d}$ indicate whether v is the current node and the destination node, respectively, $\mathbb{I}_{m_i \in v}$ indicates whether m_i is deployed on v, $\bar{d}_v(t)$ is the node degree, $\mu_v(t)$ is the compute utilization, $\xi_v(t)$ is the computing capacity discount factor, and $\ell_v(t)$ is the compute-load state. The bar notation denotes a normalized value obtained using the corresponding numerical range. Additionally, we encode the request context as $\mathbf{c}_{r,i} = [\bar{L}_r, \bar{i}, \overline{L_r - i}, \bar{C}_{m_i}, \bar{B}_{r,i}, \bar{B}_r^{\text{rem}}]$, encompassing the chain length, $\bar{L}_r$, current stage index, $\bar{i}$, remaining stages, $\overline{L_r - i}$, required CPU cycles of m_i, $\bar{C}_{m_i}$, current-stage input data volume, $\bar{B}_{r,i}$, and remaining data volume to be routed, $\bar{B}_r^{\text{rem}}$. The request context is then projected into a latent embedding $\mathbf{z}_{r,i} = \sigma(\mathbf{W}_c \mathbf{c}_{r,i})$.

The GNN performs message passing over the active ISL graph to compute node embeddings. Let $\mathbf{h}_v^{(0)} = \sigma(\mathbf{W}_0 \mathbf{x}_v)$. At the l-th GNN layer, each satellite v updates its representation by combining its current embedding with the mean embedding of its one-hop neighbors:

$$\mathbf{h}_v^{(l+1)} = \sigma\left(\mathbf{W}_l \left[\mathbf{h}_v^{(l)}, \frac{1}{|\mathcal{N}_v|} \sum_{u \in \mathcal{N}_v} \mathbf{h}_u^{(l)}\right]\right). \tag{26}$$

where $\mathcal{N}_v$ is the set of neighboring satellites of v, $\sigma(\cdot)$ is the activation function, and $\mathbf{W}_c$, $\mathbf{W}_0$, and $\mathbf{W}_l$ are trainable projection or transformation matrices. The final embedding for node v is denoted by $\mathbf{h}_v$.

b) Action Space: Microservices may possess disparate numbers of replicas, leading to a variable scale of action sets for the serving satellite selection. For execution stage i, ELARA defines the legal action set for state $s_{r,i}$ as the set of satellites currently hosting replicas of the requested microservice m_i:

$$\mathcal{A}(s_{r,i}) = \mathcal{R}_{m_i}(t) = \{v \in \mathcal{V} \mid x_{m_i,v}(t) = 1\} \tag{27}$$

Instead of employing a fixed-dimensional output layer over all satellites, ELARA utilizes a shared candidate scorer that independently evaluates each satellite $v \in \mathcal{A}(s_{r,i})$ and performs normalization exclusively over the current legal action set.

c) Reward Function Design: Referring to the weighted scalar objective in (22), the reward function transforms the multi-objective latency-energy tradeoff into a scalar learning signal for node selection. Specifically, after executing stage i, ELARA assigns a dense local reward:

$$r_{r,i}^{\text{loc}} = -(\alpha T_{r,i}^{\text{step}} + \beta \tilde{E}_{r,i}^{\text{step}} + \lambda N_{r,i}^{\text{cross}}) \tag{28}$$

where $T_{r,i}^{\text{step}}$ is the step-wise latency including routing, communication queuing, computation waiting, and execution times; $\tilde{E}_{r,i}^{\text{step}}$ denotes the normalized communication and computation energy consumption; and $N_{r,i}^{\text{cross}}$ penalizes the number of topology slots crossed to encourage timely task completion.

Upon episode termination, the terminal signal is discounted and assigned to each stage transition, yielding the augmented reward used for PPO update:

$$\hat{r}_{r,i} = r_{r,i}^{\text{loc}} + \omega_r \frac{\gamma^{L-i} r_r^{\text{term}}}{L} \tag{29}$$

where $r_r^{\text{term}} = -(\alpha T_r^{\text{e2e}} + \beta \tilde{E}_r^{\text{tot}})$ for a successfully completed request, $r_r^{\text{term}} = -P_{\text{fail}}$ if the request violates its deadline or encounters an routing failure, and ω_r is the discount factor.

d) PPO Policy and Update Mechanism: For each candidate $v \in \mathcal{A}(s_{r,i})$, ELARA constructs a 4-dimensional feature vector $\mathbf{e}_v$, including the normalized routing delay from the current satellite $u_{r,i}$ to v, the risk of future link unavailability, the bottleneck ISL capacity gap, and the estimated computing cost on candidate node v. These features are computed using the routing and computing models described in Sec. III. The policy network uses a multi-layer perceptron (MLP) to score each candidate:

$$g_\theta(v) = \text{MLP}_\theta\big([\mathbf{h}_v, \mathbf{h}_{u_{r,i}}, \mathbf{h}_{v_d}, \mathbf{z}_{r,i}, \mathbf{e}_v]\big), \tag{30}$$

where $\mathbf{z}_{r,i}$ is the embedded request context vector. A masked softmax is applied to ensure that action probabilities are distributed over legal candidates:

$$\pi_\theta(v|s_{r,i}) = \frac{\exp(g_\theta(v))}{\sum_{v' \in \mathcal{A}(s_{r,i})} \exp(g_\theta(v'))} \tag{31}$$

The value head $V_\phi(s_{r,i})$ estimates the expected return from the current state. Using the generalized advantage estimate $\hat{A}_j$, ELARA updates the actor and critic networks by minimizing the following PPO loss function:

$$\begin{aligned}\mathcal{L}(\theta, \phi) = &- \mathbb{E}_j\Big[\min(\rho_j \hat{A}_j, \bar{\rho}_j \hat{A}_j)\Big] \\ &+ c_v \mathbb{E}_j\Big[(V_\phi(s_j) - \hat{V}_j)^2\Big] - \eta_H \mathbb{E}_j[\mathcal{H}_j],\end{aligned} \tag{32}$$

where j is the index of a transition sampled from the PPO rollout buffer, $\rho_j = \pi_\theta(a_j|s_j)/\pi_{\theta_{\text{old}}}(a_j|s_j)$ is the original probability ratio, and $\bar{\rho}_j = \text{clip}(\rho_j, 1-\epsilon, 1+\epsilon)$ is the clipped ratio. $\hat{V}_j$ is the value target and $\mathcal{H}_j = \mathcal{H}(\pi_\theta(\cdot|s_j))$ is the entropy term that encourages policy exploration.

Algorithm 1 summarizes the online procedure of node selection and data routing.

Algorithm 1: ELARA Node Selection and Data Routing

Input : Request r, attributed graph snapshots $\{\mathcal{G}^a(t)\}$, trained policy π_θ
Output: Serving nodes and routing paths

$u \leftarrow v_s$, $\tau \leftarrow \tau_{\text{in}}$, $T_r \leftarrow 0$, $E_r \leftarrow 0$;
for $i = 1$ **to** L **do**
  Construct state $s_{r,i}$ and action space $\mathcal{A}(s_{r,i})$;
  Encode $\mathcal{G}^a(t)$ and request features by GNN;
  Select $v_{r,i} \sim \pi_\theta(\cdot|s_{r,i})$ during training, or $\arg\max_v \pi_\theta(v|s_{r,i})$ during inference;
  if $v_{r,i}$ *is invalid* **then**
    terminate request with failure penalty;
  end
  Execute $u \xrightarrow{B_{r,i}} v_{r,i}$ by min-cost max-flow routing;
  if *routing fails* **then**
    terminate request with failure penalty;
  end
  Execute m_i on $v_{r,i}$ and record $T^{\text{step}}_{r,i}$, $E^{\text{step}}_{r,i}$;
  Store transition $(s_{r,i}, v_{r,i}, \hat{r}_{r,i})$ for PPO update;
  $u \leftarrow v_{r,i}$, $\tau \leftarrow$ completion time of stage i;
end
Route final output from u to v_d and compute T^{e2e}_r, E^{tot}_r;
Distribute terminal reward and update PPO with (32);

B. Min-Cost Max-Flow Data Routing

For each selected source-destination pair (u, v), ELARA routes data over a finite horizon of future topology slots. In each slot t, ELARA constructs a directed residual graph based on the current topology $\mathcal{G}(t)$. For an edge $e = (a, b)$, the residual transmission capacity within the slot is given by:

$$C_e(t) = \left[R_e(t)\left(\Delta_t - \nu^{\text{pr}}_e(t) - \nu^{\text{qe}}_e(t) - \nu^{\text{sw}}\right)\right]^+, \tag{33}$$

where Δ_t is the remaining time until the next topology update and ν^{sw} is the fixed topology switching overhead. The unit flow cost on edge m is defined as:

$$c_e(t) = \alpha(\nu^{\text{tr}}_e + \nu^{\text{pr}}_e + \nu^{\text{qe}}_e + \nu^{\text{sw}}) + \beta\widetilde{E}^{\text{tr}}_e + \rho\Gamma_e(t), \tag{34}$$

where $\Gamma_e(t)$ is the link unavailability risk, estimated by checking the link's availability in subsequent topology slots.

ELARA repeatedly finds the minimum-cost augmenting path in the residual graph and augments the maximum feasible flow along that path, subject to edge and path capacities. This process continues until the data is fully delivered or the maximum number of augmentations $K_{\max}$ if reached. The resulting positive flows are then decomposed into at most $K_{\max}$ paths per slot. For each slot, the routing delay is determined by the slowest path, while the energy consumption is the sum over all used paths, which are calcuated by (16) in Sec. III.

If multiple feasible paths exist, ELARA splits traffic among them; otherwise, it naturally reduces to a single path. Importantly, ELARA does not assume that each data transfer completes within a single topology slot. If only part of the data is delivered in slot t, the residual data is carried to slot $t+1$ and re-routed on the new topology snapshot. This residual update mechanism enables ELARA to handle partial deliveries, topology changes, and route adaption across multiple slots. The routing module also records key information for each transfer, including delay components, bottleneck capacity, delivered data volume, communication energy, and predicted link unavailability. These records are subsequently used as candidate feature components by the node selection agent.

C. Bandit-Based Service Deployment

The replica redeployment module periodically updates the service replicas every I_{dep} time slots. It first summarizes recent request executions and computes a service pressure score for each microservice m:

$$\psi_m = n_m + \omega_1\bar{T}^{\text{route}}_m + \omega_2\bar{T}^{\text{wait}}_m + \omega_3 F_m + \omega_4 U_m, \tag{35}$$

where n_m is the invocation count, $\bar{T}^{\text{route}}_m$ is average routing delay to service m, $\bar{T}^{\text{wait}}_m$ is average compute waiting time, F_m is the number of routing failures associated with m, and U_m is replica-utilization imbalance. In each deployment round, ELARA first selects the top-K_{ms} microservices with the highest pressure scores and generates legal actions for them.

To avoid a large action space at the individual satellite level, ELARA defines actions at the orbital-plane level using a multi-armed bandit framework:

$$a = (\text{type}, m, p_s, p_t), \quad \text{type} \in \{\text{add}, \text{remove}, \text{move}, \text{no-op}\}. \tag{36}$$

After selecting an arm, ELARA maps it to specific satellites. For removal or migration operations, it prioritizes replicas that have experienced repeated failures or are connected by poor links. For addition or movement, target satellites are chosen from orbital planes that have sufficient storage capacity and relatively low service load, while avoiding nodes with low-rate neighboring links in the current topology. The migration cost is calculated using the routing module constructed in Sec. III, since service images must be transferred over ISLs:

$$C_a = \alpha T^{\text{mg}}_a + \beta\widetilde{E}^{\text{mg}}_a + \lambda N^{\text{cross}}_a + \rho\Gamma_a + \omega_z z_m + \omega_s T^{\text{start}}_m. \tag{37}$$

The estimated reward for each arm is:

$$R_a = \text{Saving}(a) + \text{Relief}(a) - C_a - \text{Penalty}(a), \tag{38}$$

where $\text{Saving}(a)$ is derived from the reduction in service pressure, $\text{Relief}(a)$ rewards the removal or relocation of problematic replicas, and $\text{Penalty}(a)$ discourages placements near low-rate links. ELARA applies an action only if its estimated reward R_a exceeds a predefined safety threshold and all replica count and storage constraints are satisfied. Arms are ranked using the Upper Confidence Bound (UCB) score:

$$\text{UCB}(a) = \bar{R}_a + c\sqrt{\frac{\ln N}{N_a}}, \tag{39}$$

where $\bar{R}_a$ is the empirical reward, N_a is the number of times arm a has been selected, and N is the total number of arm selections. Execution feedback is subsequently used to update the statistics of the corresponding arm.

Algorithm 2 summarizes this deployment procedure.

Algorithm 2: ELARA Bandit-Based Service Deployment

Input : Recent requests and execution results, deployment $\mathbf{X}(t)$, arm statistics
Output: Updated deployment $\mathbf{X}(t+1)$

Compute pressure scores $\{\psi_m\}$ from recent execution feedback;
Select services with the highest pressure scores and generate orbital-level arms;
for *each arm ranked by failure priority, pressure, and UCB* **do**
  Resolve the arm to concrete source and target satellites;
  if *replica-count or storage constraint is violated* **then**
    continue;
  end
  Estimate migration route, migration delay, and energy cost;
  Compute estimated reward R_a;
  if R_a *exceeds safety margin* **then**
    Apply add/remove/move operation to $\mathbf{X}(t)$;
    Update deployment-by-node records and arm statistics;
  end
end
Use subsequent execution feedback to update empirical arm rewards;

VI. Conclusion

References

[1] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of leo satellite constellations: A starlink-based measurement from end users," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pp. 1–10, 2023.

[2] H. Al-Hraishawi, H. Chougrani, S. Kisseleff, E. Lagunas, and S. Chatzinotas, "A survey on nongeostationary satellite systems: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 101–132, 2023.

[3] B. Zhang, Y. Wu, B. Zhao, J. Chanussot, D. Hong, J. Yao, and L. Gao, "Progress and challenges in intelligent remote sensing satellite systems," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 1814–1822, 2022.

[4] Z. Zhai, L. Zeng, T. Ouyang, S. Yu, Q. Huang, and X. Chen, "Seco: Multi-satellite edge computing enabled wide-area and real-time earth observation missions," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 2548–2557, 2024.

[5] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Á. Vázquez, P. Popovski, and B. Soret, "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 6180–6194, 2023.

[6] Q. Ouyang, N. Ye, J. Gao, A. Wang, and L. Zhao, "Joint in-orbit computation and communication for minimizing download time from leo satellites," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3950–3963, 2024.

[7] Z. Shen, J. Jin, C. Tan, A. Tagami, S. Wang, Q. Li, Q. Zheng, and J. Yuan, "A survey of next-generation computing technologies in space-air-ground integrated networks," *ACM Comput. Surv.*, vol. 56, aug 2023.

[8] A. Ruiz-Zafra, J. Pigueiras-del Real, M. Noguera, L. Chung, D. G. Barres, and K. Benghazi, "Servitization of customized 3d assets and performance comparison of services and microservices implementations," *IEEE Transactions on Services Computing*, vol. 17, no. 1, pp. 194–208, 2024.

[9] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3869–3886, 2016.

[10] J. Cao, S. Zhang, Q. Chen, H. Wang, M. Wang, and N. Liu, "Computing-aware routing for leo satellite networks: A transmission and computation integration approach," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 16607–16623, 2023.

[11] Z. Yu, Y. Jiang, X. Liu, Y. Shi, C. Jiang, and L. Kuang, "Microservice deployment in space computing power networks via robust reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 25, no. 2, pp. 2382–2398, 2026.

[12] Q. Xia, G. Wang, Z. Xu, W. Liang, and Z. Xu, "Efficient algorithms for service chaining in nfv-enabled satellite edge networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5677–5694, 2024.

[13] X. He, T. Wang, Y. Shi, and X. Liu, "Throughput-optimized service routing for microservice flows in leo satellite networks," *IEEE Transactions on Mobile Computing*, vol. 25, no. 5, pp. 7196–7209, 2026.

[14] D. Ron, F. A. Yusufzai, S. Kwakye, S. Roy, N. Sastry, and V. K. Shah, "Time- dependent network topology optimization for leo satellite constellations," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pp. 1–10, 2025.

[15] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge computing," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 1016–1028, 2024.

[16] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Network*, vol. 34, no. 3, pp. 224–231, 2020.

[17] X. Zhang, J. Liu, R. Zhang, Y. Huang, J. Tong, N. Xin, L. Liu, and Z. Xiong, "Energy-efficient computation peer offloading in satellite edge computing networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, pp. 3077–3091, 2024.

[18] W. Liu, H. Yang, and J. Li, "Multi-functional time expanded graph: A unified graph model for communication, storage and computation for dynamic networks over time," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 418–431, 2023.

[19] Y. Li, Q. Zhang, H. Yao, R. Gao, X. Xin, and F. R. Yu, "Stigmergy and hierarchical learning for routing optimization in multi-domain collaborative satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 5, pp. 1188–1203, 2024.

[20] Y. Gao, Z. Yan, K. Zhao, T. de Cola, and W. Li, "Joint optimization of server and service selection in satellite-terrestrial integrated edge computing networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 2, pp. 2740–2754, 2024.